Robustness of deep models

Sabanci University

Intelligent Systems Lab.

March 2026

Short-cut learning [3]

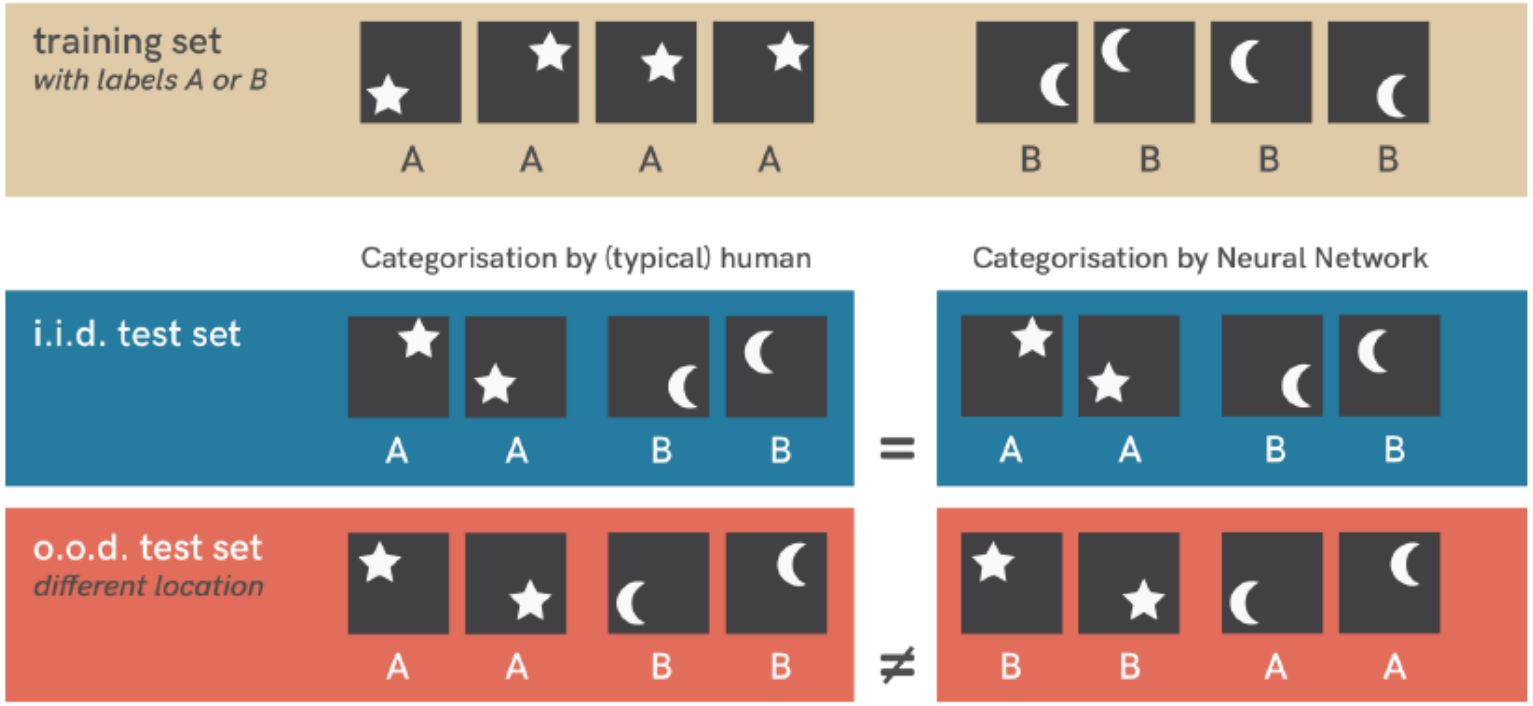
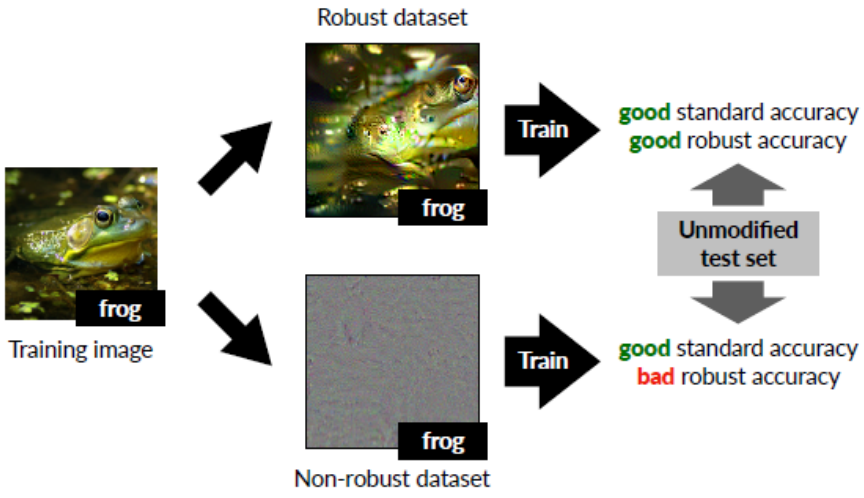
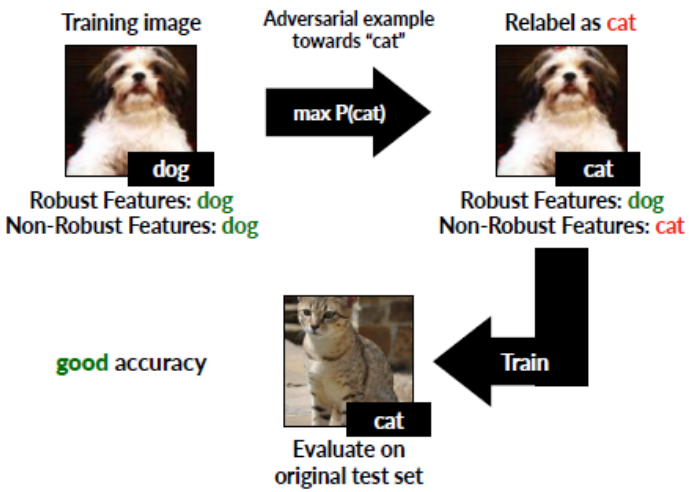


Figure 1: When trained on a simple dataset of stars and moons (top row), a standard neural network (three layers, fully connected) can easily categorise novel similar exemplars (mathematically termed i.i.d. test set). However, testing it on a slightly different dataset (o.o.d. test set, bottom row) reveals a shortcut strategy: The network has learned to associate object location with a category. During training, stars were always shown in the top right or bottom left of an image; moons in the top left or bottom right.

Adversarial examples are not bugs, they are features [5]



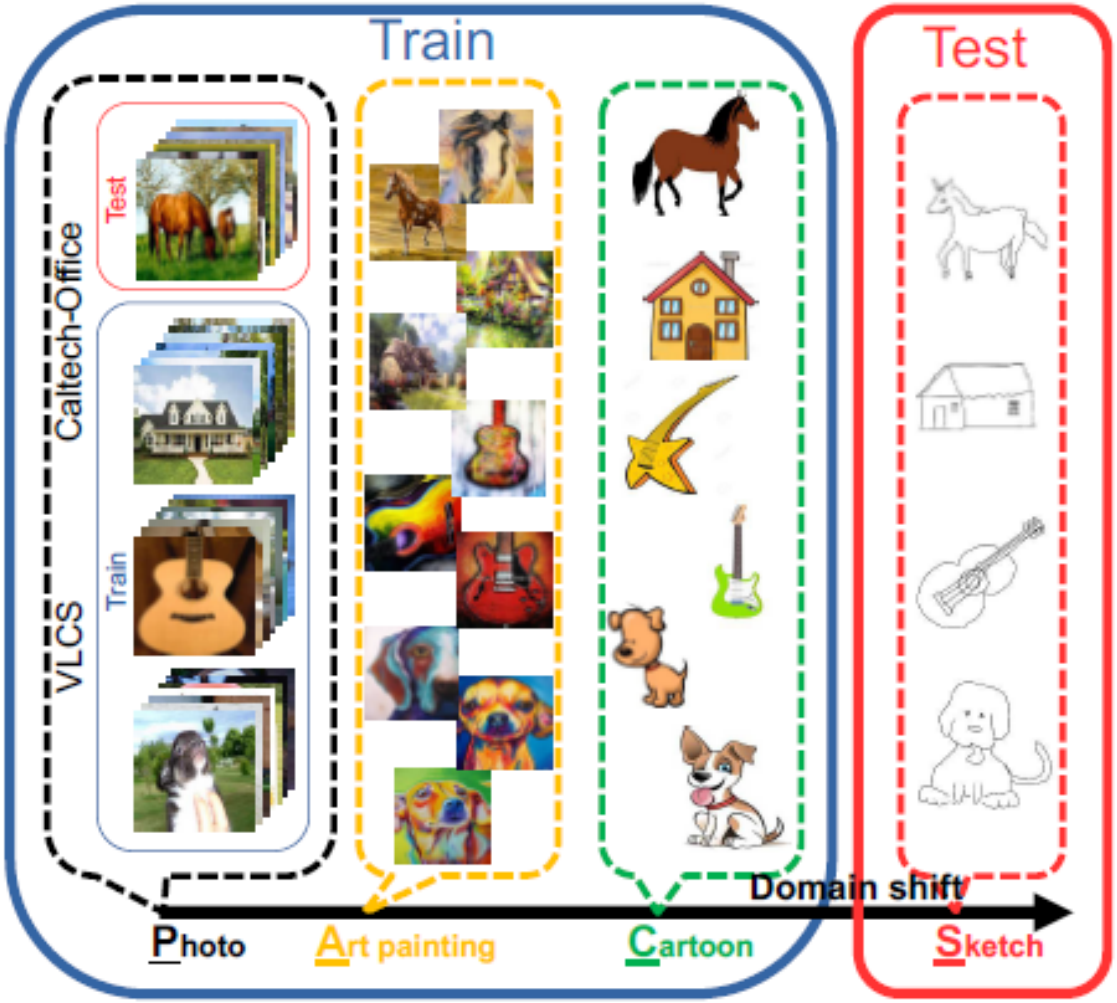
(a)



(b)

Notice: Having high test accuracy does not implies robust learning. Deep models learn features, but features may not be **robust**.

Domain shift



Same same but different :) How can we learn the semantic?

Class-activation mapping

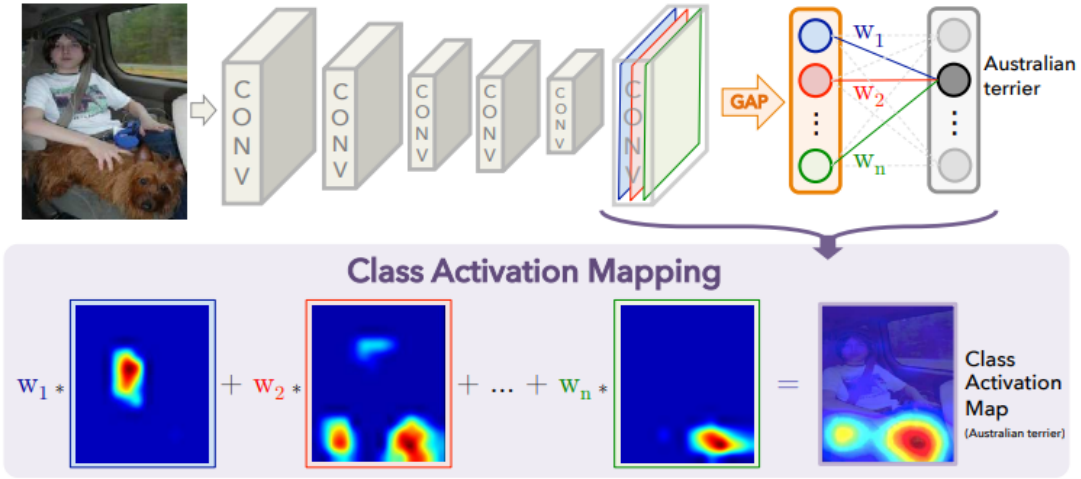


Figure 2: Illustration of class-activation mapping (CAM)

For a given image, let $f_k(x, y)$ represent the activation of unit k in the last convolutional layer at spatial location (x, y). Then, for unit k, the result of performing global average pooling is

$$F_k = \sum_{x,y} f_k(x, y).$$

Class-activation mapping

Figure 3: Illustration of class-activation mapping (CAM)

Thus, for a given class c, the input to the softmax, S_c, is

$$S_c = \sum_k w_k^c F_k$$

where w_k^c is the weight corresponding to class c. Essentially, w_k^c indicates the importance of F_k for class c.

Class-activation mapping

Figure 4: Illustration of class-activation mapping (CAM)

Finally the output of the softmax for class c, P_c, is given by

$$P_c = \frac{\exp(S_c)}{\sum_c \exp(S_c)}.$$

Class-activation mapping

Notice By plugging $F_k = \sum_{x,y} f_k(x,y)$ into the class score, S_c, we obtain

$$S_c = \sum_k w_k^c \sum_{x,y} f_k(x,y) = \sum_{x,y} \sum_k w_k^c f_k(x,y). \tag{1}$$

We define M_c as the class activation map for class c, where each spatial element is given by

$$M_c(x,y) = \sum_k w_k^c f_k(x,y). \tag{2}$$

Thus, $S_c = \sum_{x,y} M_c(x,y)$, and hence $M_c(x,y)$ directly indicates the importance of the activation at spatial grid (x,y) leading to the classification of an image to class c.

GradCam [6]

- **Step 1 — Forward Pass**
  Feed the input image $\mathbf{x}$ through the network. Let y^c denote the (pre-softmax) score for target class c. Let

  $$A^k \in \mathbb{R}^{H \times W}, \quad k = 1, \ldots, K$$

  denote the K feature maps produced by the final convolutional layer, where H and W are the spatial dimensions.
- **Step 2 — Compute Gradients**
  Backpropagate the gradient of y^c with respect to each feature map A^k:

  $$\frac{\partial y^c}{\partial A^k_{ij}}, \quad i \in \{1, \ldots, H\},\ j \in \{1, \ldots, W\}.$$

  This gradient quantifies how much a change at spatial location (i, j) in feature map k would affect the class score.

GradCam [6]

- **Step 3 — Global Average Pooling of Gradients**
  Compute a scalar importance weight α_k^c for each feature map by globally average-pooling the gradients:

$$\boxed{\alpha_k^c = \frac{1}{Z} \sum_{i=1}^{H} \sum_{j=1}^{W} \frac{\partial y^c}{\partial A_{ij}^k}}$$

  where $Z = H \times W$ is a normalisation constant. Intuitively, α_k^c captures the **global importance** of feature map k for class c.
- **Step 4 — Weighted Combination and ReLU**
  Form the GradCAM localisation map $L^c \in \mathbb{R}^{H \times W}$ as a weighted combination of the feature maps, followed by a ReLU:

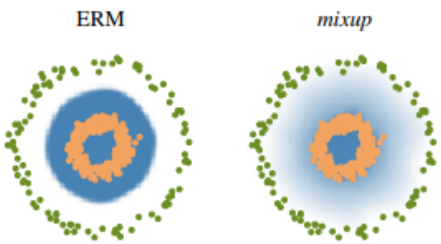
$$\boxed{L_{\text{GradCAM}}^c = \text{ReLU}\left(\sum_{k=1}^{K} \alpha_k^c \, A^k \right)}$$

  The ReLU retains only those activations that have a *positive* influence on the target class.

GradCam [6]

```
# y1, y2 should be one-hot vectors
for (x1, y1), (x2, y2) in zip(loader1, loader2):
    lam = numpy.random.beta(alpha, alpha)
    x = Variable(lam * x1 + (1. - lam) * x2)
    y = Variable(lam * y1 + (1. - lam) * y2)
    optimizer.zero_grad()
    loss(net(x), y).backward()
    optimizer.step()
```

(a) One epoch of *mixup* training in PyTorch.



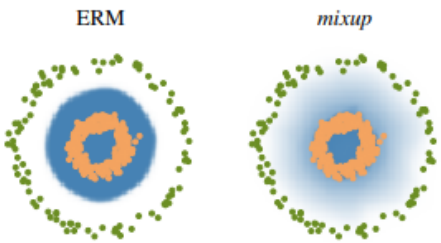
(b) Effect of *mixup* ($\alpha = 1$) on a toy problem. Green: Class 0. Orange: Class 1. Blue shading indicates $p(y = 1|x)$.

Recall: Previously we have discussed, for training DNNs, we use ERM, since we do not know the true data distribution, which may result in overfitting.

Data Augmentation: MixUp [8]

```
# y1, y2 should be one-hot vectors
for (x1, y1), (x2, y2) in zip(loader1, loader2):
    lam = numpy.random.beta(alpha, alpha)
    x = Variable(lam * x1 + (1. - lam) * x2)
    y = Variable(lam * y1 + (1. - lam) * y2)
    optimizer.zero_grad()
    loss(net(x), y).backward()
    optimizer.step()
```

(a) One epoch of *mixup* training in PyTorch.



(b) Effect of *mixup* ($\alpha = 1$) on a toy problem. Green: Class 0. Orange: Class 1. Blue shading indicates $p(y = 1|x)$.

MixUp constructs virtual training examples

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \text{where } x_i, x_j \text{ are raw input vectors} \tag{1}$$

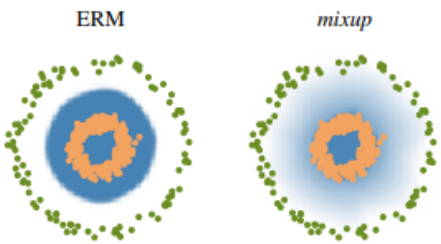
$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j, \quad \text{where } y_i, y_j \text{ are one-hot label encodings} \tag{2}$$

(x_i, y_i) and (x_j, y_j) are two examples drawn at random from our training data, and $\lambda \in [0, 1]$.

Data Augmentation: MixUp [8]

```
# y1, y2 should be one-hot vectors
for (x1, y1), (x2, y2) in zip(loader1, loader2):
    lam = numpy.random.beta(alpha, alpha)
    x = Variable(lam * x1 + (1. - lam) * x2)
    y = Variable(lam * y1 + (1. - lam) * y2)
    optimizer.zero_grad()
    loss(net(x), y).backward()
    optimizer.step()
```

(a) One epoch of *mixup* training in PyTorch.



(b) Effect of *mixup* ($\alpha = 1$) on a toy problem. Green: Class 0. Orange: Class 1. Blue shading indicates $p(y = 1|x)$.

Recall: Previously we have discussed, for training DNNs, we use ERM, since we do not know the true data distribution, which may result in overfitting.

Data Augmentation: CutMix [7]

	ResNet-50	Mixup [48]	Cutout [3]	CutMix
Image				
Label	Dog 1.0	Dog 0.5 Cat 0.5	Dog 1.0	Dog 0.6 Cat 0.4
ImageNet Cls (%)	76.3 (+0.0)	77.4 (+1.1)	77.1 (+0.8)	**78.6** **(+2.3)**
ImageNet Loc (%)	46.3 (+0.0)	45.8 (-0.5)	46.7 (+0.4)	**47.3** **(+1.0)**
Pascal VOC Det (mAP)	75.6 (+0.0)	73.9 (-1.7)	75.1 (-0.5)	**76.7** **(+1.1)**

Figure 5: Comparison of different augmentation methods.

Data Augmentation: CutMix [7]

- Let $x \in \mathbb{R}^{W \times H \times C}$ and y denote a training image and its label, respectively.
- CutMix generate a new training sample $(\tilde{x}, \tilde{y})$ by combining two training samples (x_A, y_A) and (x_B, y_B).
- The generated training sample $(\tilde{x}, \tilde{y})$ is used to train the model with its original loss function.
- We define the combining operation as

$$\tilde{x} = \mathbf{M} \odot x_A + (\mathbf{1} - \mathbf{M}) \odot x_B \tag{3}$$

$$\tilde{y} = \lambda y_A + (1 - \lambda) y_B \tag{4}$$

  where $\mathbf{M} \in \{0, 1\}^{W \times H}$ denotes a binary mask indicating where to drop out and fill in from two images, $\mathbf{1}$ is a binary mask filled with ones, and $\odot$ is element-wise multiplication.
- λ is sampled from the beta distribution $\text{Beta}(\alpha, \alpha)$.

Data Augmentation: CutMix [7]

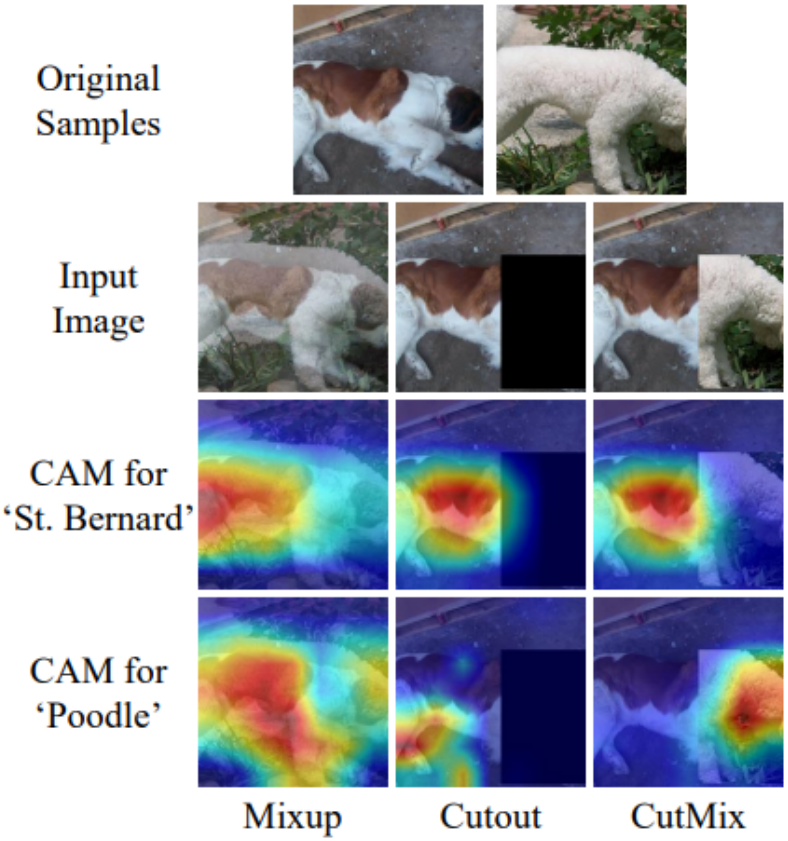


Figure 6: Illustration of CAM for different augmentations

Notice: CutMix helps network to focus on class-specific features.

Data Augmenttaion : AutoAugment [1]

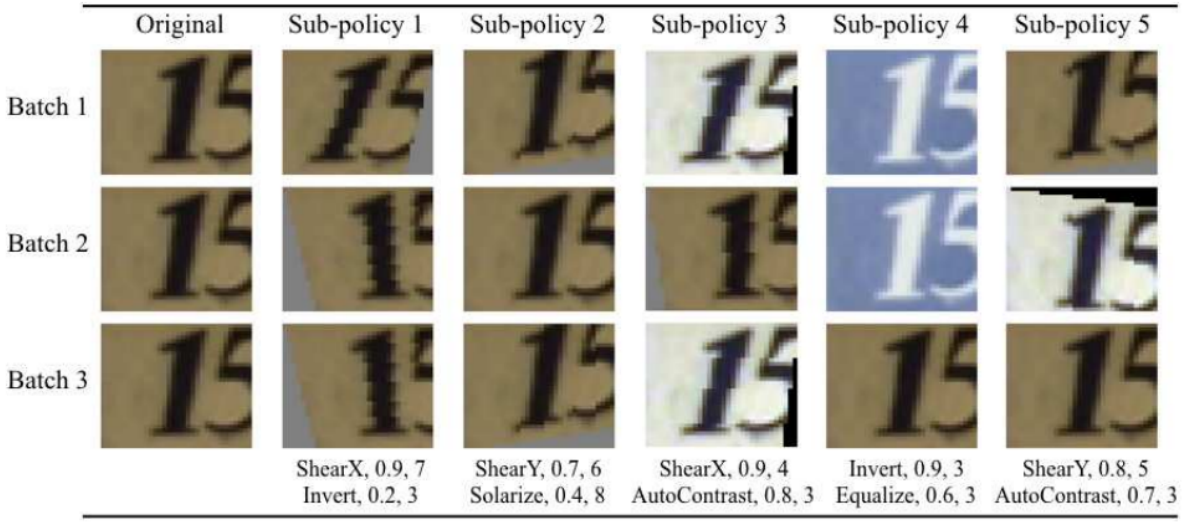


Figure 7: Illustration of sub-policies.

Search space: A policy consists of 5 sub-policies with each sub-policy consisting of two image operations to be applied in sequence. Additionally, each operation is also associated with two hyperparameters: 1) the probability of applying the operation, and 2) the magnitude of the operation.

Data Augmentation AutoAugment [1]

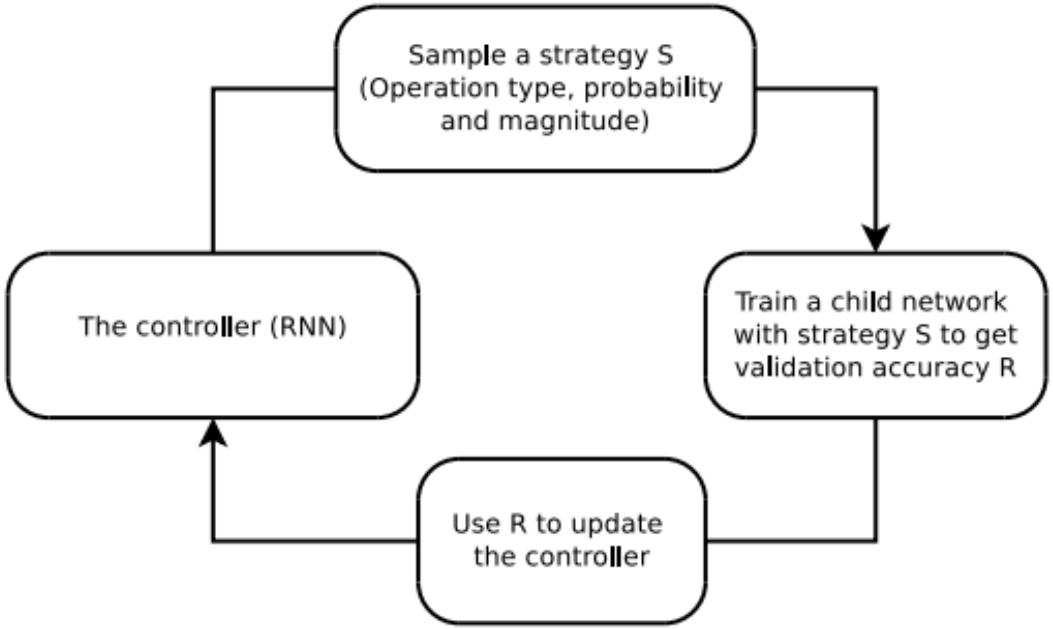


Figure 8: Illustration of sequence of transformations in Randaugment.

- Use a search method (e.g., Reinforcement Learning) to seek better data augmentation policies.
- A controller RNN predicts an augmentation policy from the search space.
- A child network with a fixed architecture is trained to convergence

Data Augmentation: Random augmentation [2]

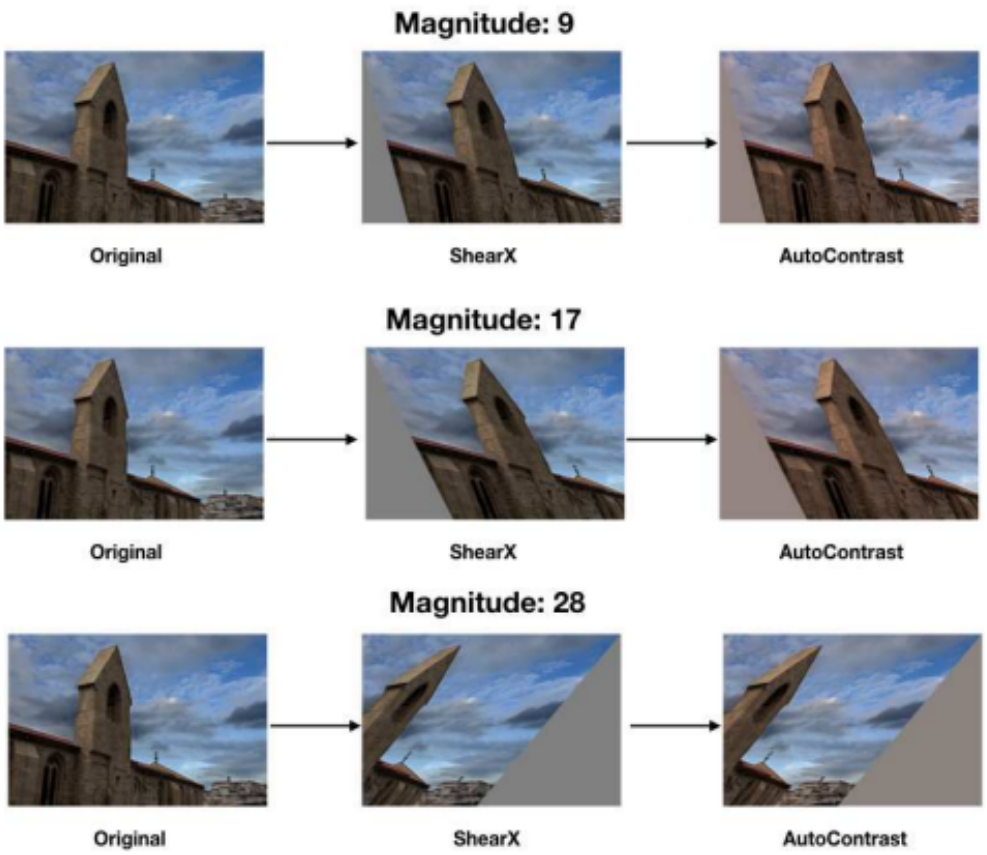


Figure 9: Illustration of sequence of transformations in Randaugment.

Data Augmentation: Random augmentation [2]

```
transforms = [
'Identity', 'AutoContrast', 'Equalize', 'Rotate',
     'Solarize', 'Color',
'Posterize', 'Contrast', 'Brightness', 'Sharpness',
     'ShearX', 'ShearY',
'TranslateX', 'TranslateY']

def randaugment(N, M):
"""Generate a set of distortions.

  Args:
    N: Number of augmentation transformations to apply
         sequentially.
    M: Magnitude for all the transformations.
"""

 sampled_ops = np.random.choice(transforms, N)
 return [(op, M) for op in sampled_ops]
```

Figure 10: Illustration of the implementation.

Notice: Randaugment has two parameters; number of transformations (N) and magnitude of transformation (M).

Data Augmentation: AugMix [4]

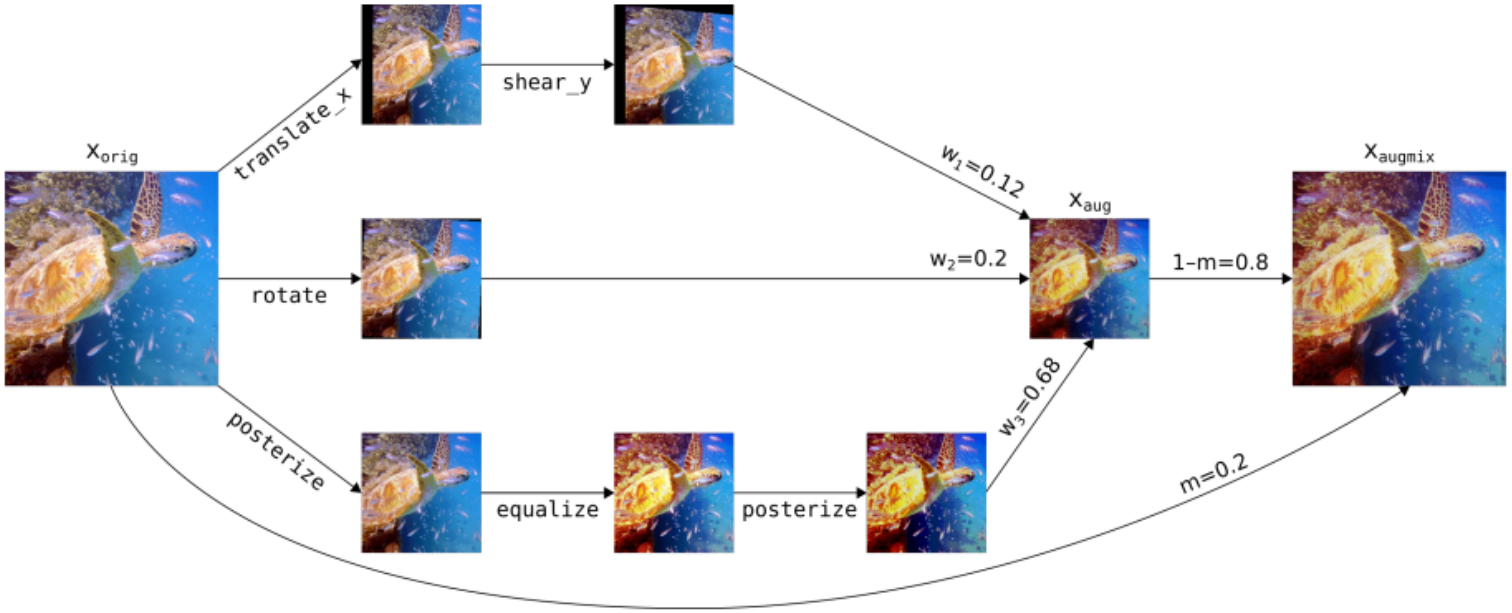


Figure 11: Illustration of generating augmented sample X_{augmix}

Notice: AugMix first combines different augmented images (with different sequence depth of transformations), then combines it with original image.

Data Augmentation: AugMix [4]

Algorithm AUGMIX Pseudocode

Input: Model $\hat{p}$, Classification Loss $\mathcal{L}$, Image x_{orig}, Operations $\mathcal{O} = \{\text{rotate}, \ldots, \text{posterize}\}$
function AugmentAndMix(x_{orig}, $k = 3, \alpha = 1$)
  Fill x_{aug} with zeros
  Sample mixing weights $(w_1, w_2, \ldots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \ldots, \alpha)$
  for $i = 1, \ldots, k$ **do**
    Sample operations $\text{op}_1, \text{op}_2, \text{op}_3 \sim \mathcal{O}$
    Compose operations with varying depth $\text{op}_{12} = \text{op}_2 \circ \text{op}_1$ and $\text{op}_{123} = \text{op}_3 \circ \text{op}_2 \circ \text{op}_1$
    Sample uniformly from one of these operations chain $\sim \{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$
    $x_{\text{aug}} \mathrel{+}= w_i \cdot \text{chain}(x_{\text{orig}})$ ▷ *Addition is elementwise*
  end for
  Sample weight $m \sim \text{Beta}(\alpha, \alpha)$
  Interpolate with rule $x_{\text{augmix}} = m x_{\text{orig}} + (1 - m) x_{\text{aug}}$
  return x_{augmix}
end function
$x_{\text{augmix1}} = \text{AugmentAndMix}(x_{\text{orig}})$ ▷ $x_{augmix1}$ *is stochastically generated*
$x_{\text{augmix2}} = \text{AugmentAndMix}(x_{\text{orig}})$ ▷ $x_{augmix1} \neq x_{augmix2}$
Loss Output: $\mathcal{L}(\hat{p}(y \mid x_{\text{orig}}), y) + \lambda\, \text{Jensen-Shannon}(\hat{p}(y \mid x_{\text{orig}}); \hat{p}(y|x_{\text{augmix1}}); \hat{p}(y|x_{\text{augmix2}}))$

Figure 12: AugMix Algorithm.

Data Augmentation: AugMix [4]

- Minimize the Jensen-Shannon divergence among the posterior distributions of the original sample x_{orig} and its augmented variants.
- For $p_{\text{orig}} = \hat{p}(y \mid x_{\text{orig}})$, $p_{\text{augmix1}} = \hat{p}(y \mid x_{\text{augmix1}})$, $p_{\text{augmix2}} = \hat{p}(y \mid x_{\text{augmix2}})$, we replace the original loss $\mathcal{L}$ with:

$$\mathcal{L}(p_{\text{orig}}, y) + \lambda \, \mathrm{JS}(p_{\text{orig}};\, p_{\text{augmix1}};\, p_{\text{augmix2}}) \tag{5}$$

Interpretation

Imagine sampling from one of the three distributions $p_{\text{orig}}, p_{\text{augmix1}}, p_{\text{augmix2}}$. The Jensen-Shannon divergence measures the average information that the sample reveals about *which* distribution it was drawn from. Minimising it forces the model to be invariant to augmentation.

Data Augmentation: AugMix [4]

- Computing JS via the average mixture $M = \dfrac{p_{\text{orig}} + p_{\text{augmix1}} + p_{\text{augmix2}}}{3}$:

$$\begin{aligned}&\mathrm{JS}(p_{\text{orig}};\, p_{\text{augmix1}};\, p_{\text{augmix2}})\\ &= \frac{1}{3}\Big(\mathrm{KL}[p_{\text{orig}} \| M] + \mathrm{KL}[p_{\text{augmix1}} \| M] + \mathrm{KL}[p_{\text{augmix2}} \| M]\Big)\end{aligned} \tag{6}$$

Reference papers I

Ekin D. Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V. Le.
Autoaugment: Learning augmentation strategies from data.
In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le.
Randaugment: Practical automated data augmentation with a reduced search space.
In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2020.

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann.
Shortcut learning in deep neural networks.
Nature Machine Intelligence, 2(11):665–673, 2020.

Reference papers II

Dan Hendrycks, Norman Mu, Ekin Dogus Cubuk, Barret Zoph, Justin Gilmer, and Balaji Lakshminarayanan.
Augmix: A simple data processing method to improve robustness and uncertainty.
In *International Conference on Learning Representations*.

Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, and Aleksander Madry.
Adversarial examples are not bugs, they are features.
Advances in neural information processing systems, 32, 2019.

Reference papers III

Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra.
Grad-cam: Visual explanations from deep networks via gradient-based localization.
In *Proceedings of the IEEE international conference on computer vision*, pages 618–626, 2017.

Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo.
Cutmix: Regularization strategy to train strong classifiers with localizable features.
In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6023–6032, 2019.

Reference papers IV

Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz.
mixup: Beyond empirical risk minimization.
In *International Conference on Learning Representations*, 2018.